

实验一

学号：22320021 姓名：陈安康

算法逻辑和实现思路

本次采用模拟退火算法来解决本次布局布线问题。

模拟退火算法（Simulated Annealing，简称 SA）是一种 概率性全局优化算法，灵感来源于金属材料的物理退火过程。在实际退火中，材料在高温下原子活动剧烈，能够跳出局部能谷，逐渐降温后趋于低能稳定状态。模拟退火正是借助这一思想，寻求复杂优化问题的 全局最优解。在芯片布局布线问题中，经常使用模拟退火算法来进行解决。

在基础的模拟退火算法之上，我实现了一些小改进，比如在程序执行过程中统计计算接受率，依据接受率来动态调整模拟退火的退火温度。同时利用允许的不平衡要求，在生成新状态时不仅实现两个节点的交换，而且实现节点的单方向移动。同时保证满足平衡条件。具体代码如下：

```
// 温度下降函数
double Solution::cooling_schedule(double t, double acc_rate) {
    if (acc_rate > ACC_RATE) {
        t = t * 0.98;
    } else {
        t = t * 0.95;
    }
    return t;
}
```

如果当前接受率比较高（ $> \text{ACC_RATE}$ ），说明系统还在积极探索阶段——也就是说，大部分新解被接受，此时可能还远离最优解 → 降温速度可以慢一点（保留探索能力）。

如果接受率较低（ $\leq \text{ACC_RATE}$ ），说明大多数新解都被拒绝，系统已经趋于稳定 → 可加快降温来收敛。

同时为了提高搜索效率，在生成新状态时，只对“边界点”进行操作。因为内部点的交换和移动都会导致解代价提高，并且并不会有助于接近更优的解，或者说可以由边界点的操作达到相同的效果。可以考虑划分 X 和 Y，节点 a 和 b 分别是 X 和 Y 的内部节点，此时若进行两者交换，或者 a(b)移动到 X(Y)，此时目前解代价会升高。从长远来看，若此次交换（移动操作）的结果属于最优解的一部分，即 ab 交换后或者 ab 移动后确实位于最优解中，那么在当前状态中肯定有一条从边界点到 a 和 b 节点的路径（否则 a 和 b 节点就是“孤立点”，对解没有任何影响），所以存在通过对边界点的操作来将 ab 节点执行成最优解节点的路径。因

此不存在理论上的局限。由此，在每次生成新节点时，只对边界节点进行操作。同时为了避免每次寻找边界节点是的时间开销，采用维护而不是重新生成的方式来更新边界节点集。

部分代码如下：

```
set<int> Solution::find_boundary_nodes(Graph &graph, set<int> &X, set<int> &Y, bool from_X) {
    set<int> boundary_nodes;
    const set<int> &group = from_X ? X : Y;
    const set<int> &other_group = from_X ? Y : X;

    for (int node_id : group) {
        Node *node = graph.get_node(node_id);
        if (!node) continue;

        // 如果它连接了至少一个来自对方集合的节点，则是边界节点
        vector<Net *> nets = node->get_nets();
        for (const auto &net : nets) {
            for (const auto &adj : net->get_nodes()) {
                if (adj->get_index() != node_id && other_group.count(adj->get_index())) {
                    boundary_nodes.insert(node_id);
                }
            }
        }
    }

    return boundary_nodes;
}
```

```
// 注意：假设index在S中操作已经完成
void Solution::update_boundary_set(Graph &graph, set<int> &S, int index, bool from_X, bool is_in) {
    set<int> &boundary = from_X ? boundary_X : boundary_Y;
    Node *node = graph.get_node(index);
    // 如果是移出操作
    if (!is_in) {
        // 先删除源节点
        if (boundary.find(index) != boundary.end()) boundary.erase(index);
        // 添加在S中的邻接点
        for (const auto &net : node->get_nets()) {
            for (const auto &adj : net->get_nodes()) {
                if (adj->get_index() != index && S.count(adj->get_index())) {
                    boundary.insert(adj->get_index());
                }
            }
        }
    } else {
        bool is_acc = false;
        for (const auto &net : node->get_nets()) {
            for (const auto &adj : net->get_nodes()) {
                if (adj->get_index() != index && !S.count(adj->get_index())) {
                    is_acc = true;
                    break;
                }
            }
        }
        if (is_acc) {
            boundary.insert(index);
            break;
        }
    }
}
```

后续比较后发现只对边界点进行操作的性能相比于全局操作进步有限，根本原因在于迭代次

数太少，两个划分中的点绝大部分都是边界点，导致和全局操作几乎没有区别，如果迭代充分，应该能取得不错的性能提升。

结果展示

由于自身电脑硬件限制，并且使用的是虚拟机，因此在执行迭代时没有办法进行完全迭代，此次所有实验是在初始温度 100，终止温度 0.1，每次温度内循环次数 500 的设置下进行的。（已经是我的电脑可接受的极限）

部分实验结果展示如下：

```
0.13
0.134
最佳割边数：4606
5841 6205
划分结果已保存到：ibm01_partition.txt
割边数：4606
评估结果：4606
```

```
0.174
0.24
最佳割边数：8026
9258 9971
划分结果已保存到：ibm02_partition.txt
割边数：8026
评估结果：8026
```

```
0.198
最佳割边数：10009
10727 11488
划分结果已保存到：ibm03_partition.txt
割边数：10009
评估结果：10009
```

对于 04 尝试采用 1000 次内循环，结果跑了 8 个小时，遂后面又改成了采用 500 次内循环。

```
0.137
最佳割边数：10466
12879 13830
划分结果已保存到：ibm04_partition.txt
割边数：10466
评估结果：10466
```

```
最佳割边数：12128
13781 14923
划分结果已保存到：ibm05_partition.txt
割边数：12128
评估结果：12128
```

```
0.1250
最佳割边数：14725
15390 16647
划分结果已保存到：ibm06_partition.txt
割边数：14725
评估结果：14725
```

Benchmark	我的解	最优解	差距
IBM01	4606	200	4406
IBM02	8026	307	7719
IBM03	10009	951	9058
IBM04	10466	573	9893
IBM05	12128	1706	10422
IBM06	14725	962	13763
IBM07	21617	878	20739
IBM08	24971	1140	23831
IBM09	30819	620	30199
IBM10	39797	1253	38544
IBM11	42922	1051	41871
IBM12	41440	1919	39521
IBM13	53657	831	52826
IBM14	71231	1842	69389
IBM15	85432	2730	82702
IBM16	89531	1827	87704
IBM17	90523	2270	92253
IBM18	98764	1521	97243

实验结果可以说是十分不理想，基本上所有的解都和最优解保持极其大的差距。经过分析，我发现主要问题有以下几点：

迭代不充分。由于我的硬件设备确实有限，温度为 100，内迭代为 500 次的一次运行都会消耗 6-7 小时，导致不敢再进行更深的迭代。导致迭代十分不充分。分析其时间消耗，主要来自代价计算和一些赋值操作。代价计算应该可以通过类似于维护边界点的方法进行维护。不必每次都进行计算。后续可以在此方面进行一些优化。

初始化操作过于简单，在代码中最初，我简单先填充 X 划分，等填充到一半后再填充 Y，虽然这样填充代码简单，但完全没有任何优化，对于依赖初始解的模拟退火算法，这无疑是毁灭性的。我后续对这种方法进行改进，包括采用随机选择节点放入某一个划分，并且将其邻近划分也放入同一个划分。由于实在没有时间进行全部的划分的测试，所以只对第一个数据集进行测试。采用这种方法的划分结果如下：

```
最佳割边数: 3301
2473 3012
划分结果已保存到: ibm01_partition.txt
割边数: 3301
评估结果: 3301
```

可以看见相比于初始的划分方式，这种划分的提升还是很明显的，但依旧没有达到最理想的情况，分析还是迭代不够导致的。

虽然实验要求不需要提交代码，但本次实验效果确实太差，故此提交代码作为实验依据。